MLDS2 · From Notebook to Production · Part 2 of 3

Experiment Tracking and Model Management

From a reproducible training command to a model an application can retrieve, with MLflow

25 minutes Lecture: 16 November 2026

1 Train | **2 Track** | 3 Serve | 4 Monitor

We already built a tracker by hand

Part 1's run folders record everything. They do not scale to a team.

OUR OUTPUTS/ FOLDER TODAY

```
2026-11-13_22-13-12Z_ridge_v1_87e9e4/
2026-11-13_22-13-27Z_random_forest_…/
    manifest.json  metrics.json  model.pkl
comparison.md   (report.py, 200 lines)
… one folder per run, one laptop
```

WHAT BREAKS WHEN A TEAM USES IT

- Search runs across people and machines
- Know which runs are comparable
- Find last month's run and its exact code
- **Hand one model to another team**

Tracking keeps the same records: shared, searchable and linked to the models they produced.

Where MLflow fits

Tracking and model management connect reproducible training to application use.

PART 1

Reproducible training

Code · data · environment · evaluation

→

MLFLOW TRACKING

Experiment tracking

Runs · parameters · metrics · artifacts

→

MLFLOW MODELS

Model management

Logged models · registry versions · aliases

→

PART 3

Application use

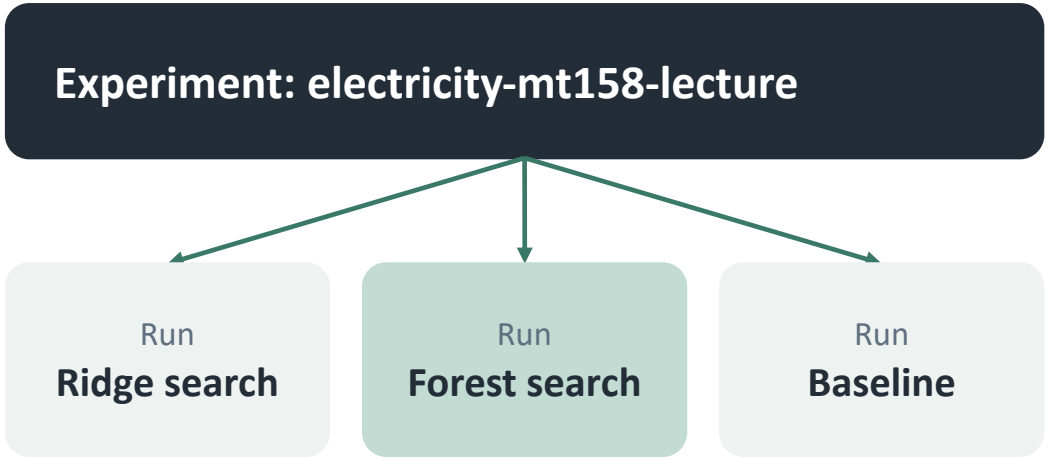
Serving · monitoring · drift

Boundary: this section ends with a versioned model we can retrieve, not a deployed service.

The same ideas apply to other trackers (e.g. Weights & Biases). We run MLflow locally; a team would share one tracking server.

Experiments, runs and what a run records

An experiment groups related runs and logged models. A run is the record of one execution.



One run can cover a single fit, several CV folds or a whole search. Choose its scope deliberately.

Parameters

Settings you choose

alpha, max_depth, seed

Metrics

Outcomes you measure

validation MAE, fold scores

Artifacts

Files you keep

plots, configs, fitted model

Tags + datasets

Searchable context

purpose, data digest, code version

Adding MLflow to existing training

Training computes the result. A thin tracking wrapper records it.

```
mlflow.set_experiment("electricity-mt158-lecture")

with mlflow.start_run(run_name="forest-depth-10"):
    mae = evaluate_cv(pipeline, folds)    # compute
    pipeline.fit(X_dev, y_dev)            # compute

    mlflow.log_params({"max_depth": 10, "min_samples_leaf": 3})
    mlflow.log_metric("cv_mae_kw", mae)
    mlflow.log_artifact("forecast.png")
    mlflow.sklearn.log_model(pipeline, name="forecast_pipeline",
                             input_example=X_dev.head())
```

1. **Open a run**

   A context manager. An exception marks the run FAILED.

2. **Compute**

   The same temporal CV and refit as before.

3. **Record**

   Code version, data identity, config, environment, metrics, files.

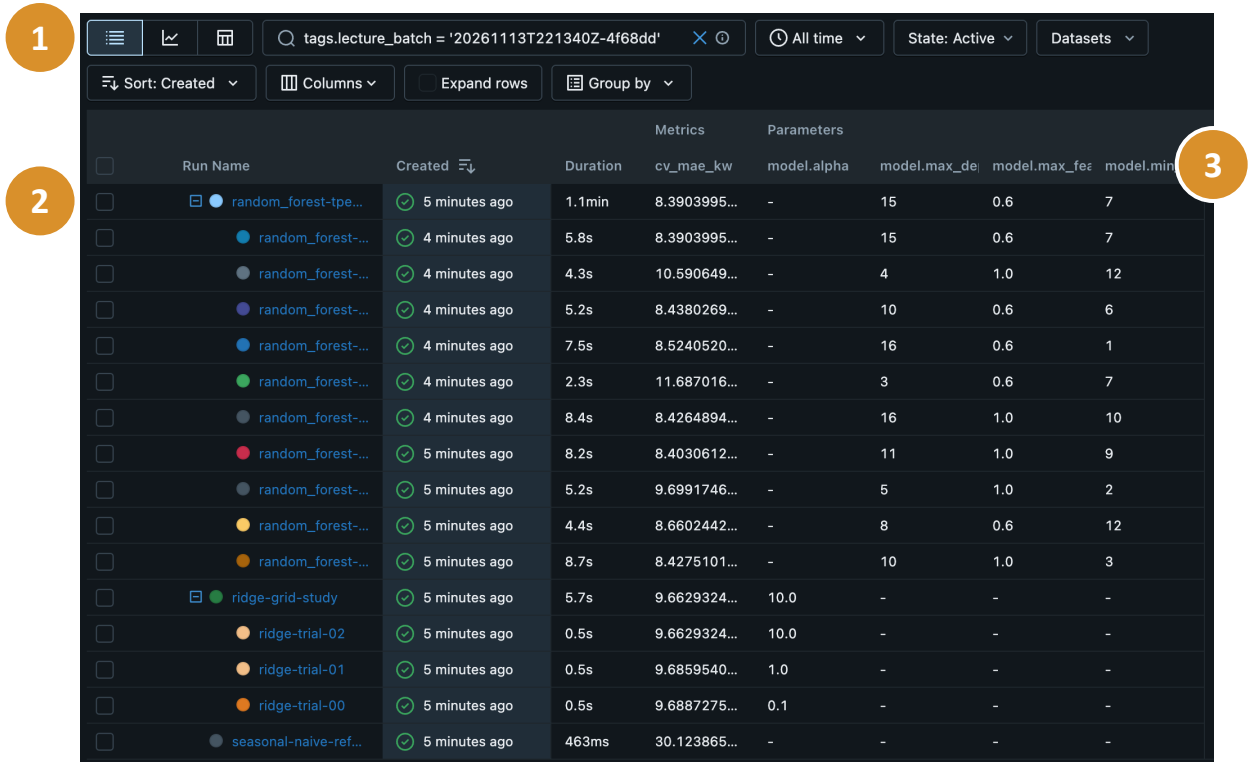
4. **Package**

   The fitted pipeline becomes a logged model.

Separate concerns: train.py never imports mlflow. tracking.py wraps it, so training still runs in tests, notebooks and CI without a tracker.

Finding runs in the Tracking UI

Filter by tags, then expand a search into its candidate runs.



Actual MLflow 3.18 Runs view of the recorded lecture batch.

1 **Filter with tags**

tags.lecture_batch = '...' keeps one batch. Tags find work among hundreds of runs.

2 **Parent and child runs**

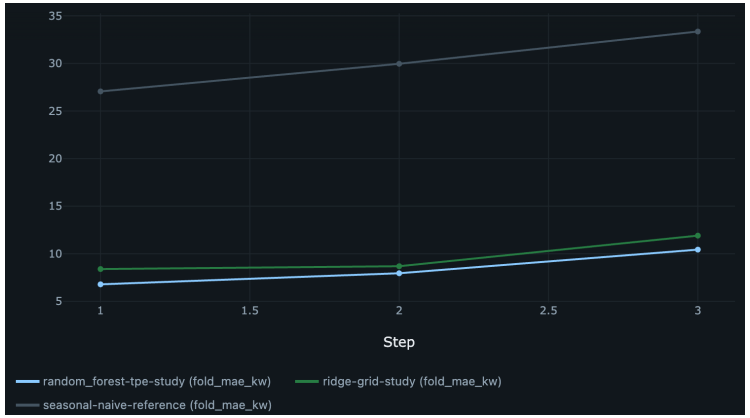
A parent is one search; each child is one candidate: 10 forest settings, 3 Ridge alphas.

3 **Settings beside outcomes**

Parameter and metric columns show what changed and what happened. A run name opens its record.

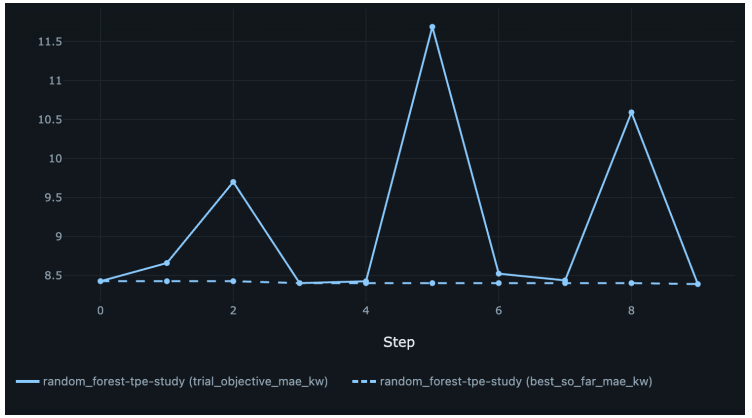
Comparing runs visually

Metrics logged with a step, and parameters per trial, become MLflow charts with no plotting code.



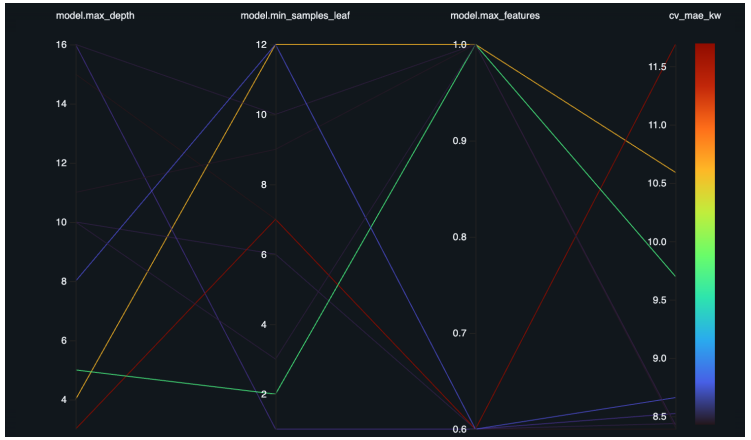
1 Fold error per run

Step = chronological fold. All runs worsen in fold 3; the baseline stays far above.



2 Optuna search progress

Step = trial. Solid: each trial's error. Dashed: best so far; most gain by trial 3, a small gain at trial 9.



3 Settings vs error

One line per forest trial. Shallow trees (depth 3–5) give the three worst errors; depth 10+ all score about 8.4.

Ten trials show search behaviour, not robust parameter importance. Folds are time windows, not epochs.

Selecting a model: compare, then decide in code

The UI is for investigating. The decision rule should be code you can rerun.

Run Name:	random_forest-tpe-study	ridge-grid-study	seasonal-naive-reference
cv_baseline_mae_kw	30.12	30.12	30.12
cv_mae_kw	8.39	9.663	30.12
comparison_group	0b86a7bd8ec082d8ba2209b	0b86a7bd8ec082d8ba2209b	0b86a7bd8ec082d8ba2209b

Same baseline error and comparison_group = comparable runs

Ranking metric: validation MAE (kW)

RANK ONLY COMPARABLE RUNS

Same data digest, same target hours, same protocol (comparison_group tag), status FINISHED.

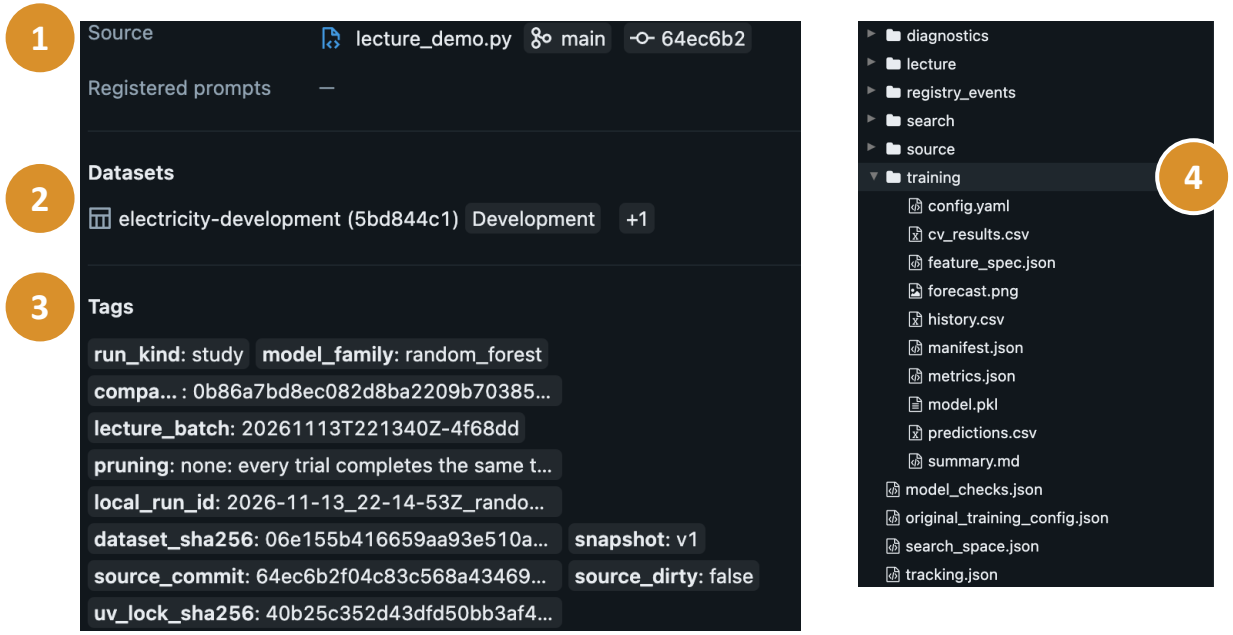
Select with code

```
runs = mlflow.search_runs(
    experiment_names=["electricity-mt158-lecture"],
    filter_string="tags.run_kind = 'study' and "
                  "attributes.status = 'FINISHED'",
    order_by=["metrics.cv_mae_kw ASC"])
best = runs.iloc[0]
assert best["tags.comparison_group"] == group
assert best["metrics.cv_mae_kw"] < baseline_mae
mlflow.register_model(best["tags.model_uri"], name)
```

Rerunnable, reviewable, CI-ready. Simplified from selection.select_candidate.

Provenance: where did this result come from?

A run links its result to code, data, environment and outputs.



1 **Source**

Script, branch and Git commit.

2 **Dataset**

Digest of the data the run used.

3 **Tags**

Data hash, lockfile hash, commit 64ec6b2 and source_dirty: false.

4 **Artifacts**

Config, predictions, plots, archived source and uv.lock.

An ID identifies a resource; it does not keep it. Recovery still needs Git, DVC and uv.

Run, logged model, registered version

Three different objects answer three different questions.

Tracked run

What was executed and measured?

`random_forest-tpe-study`

Logged model

What exactly was fitted, and how do I load it?

`forecast_pipeline`

Registered version

Which model does the team refer to?

`electricity-mt158-lecture/v2`

Registration creates a named, numbered reference. It is not approval and not a deployment.

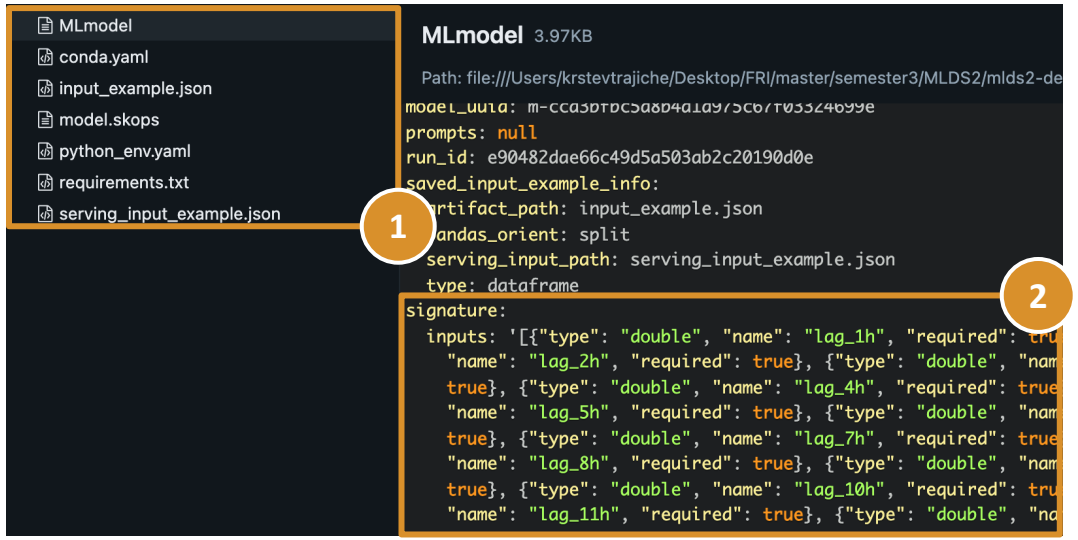
In our demo the registered model shares the experiment's name. They are still different objects.

The model contract: what a consumer receives

Another developer gets a reference, an interface and an environment, and must know what is not inside.

HANDED TO THE APPLICATION

Reference	`models:/electricity-mt158-lecture/2`
Package	Fitted scaling + encoding + estimator
Input	29 engineered numeric columns
Output	Next-hour load in kW
Environment	requirements.txt (no training tools)



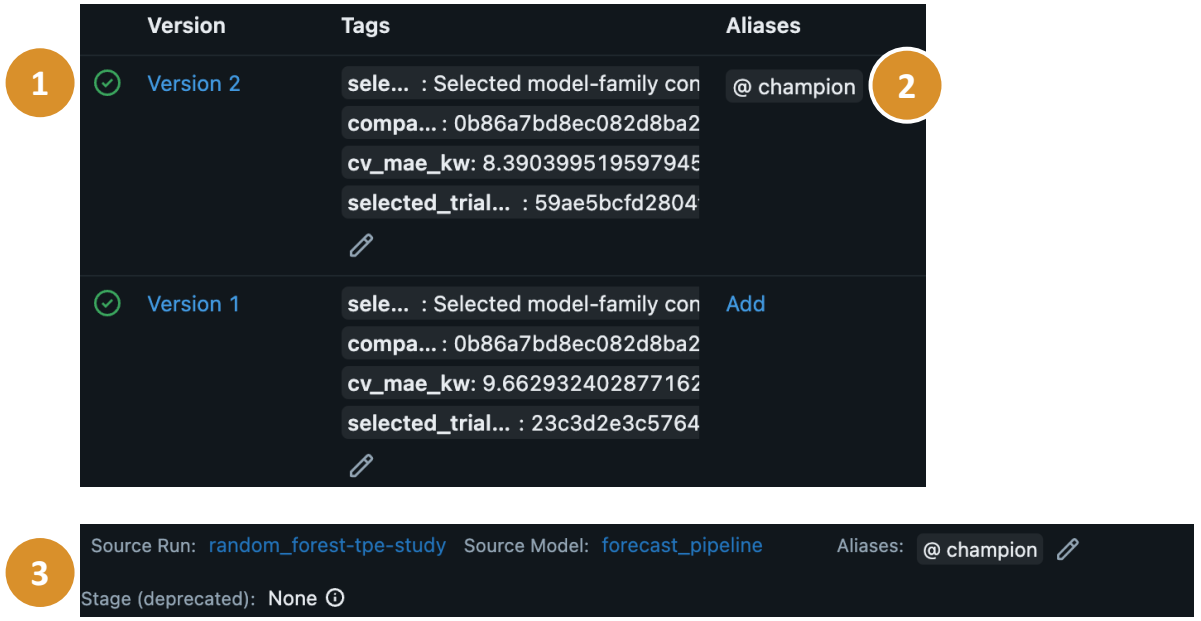
NOT INSIDE THE MODEL: FEATURES BUILT FROM RAW METER HISTORY

Option A · features inside the model
A custom pyfunc wraps the feature code; consumers send raw history.

Option B · shared feature library (our demo)
Consumers call build_next_features, the same feature code training uses.

Registry and aliases: the hand-off boundary

Data scientists publish versions; applications load a name the team controls.



1 Numbered versions never change **2** champion → version 2 **3** lineage to run and logged model

Consumers load by alias

```
uri = "models:/electricity-mt158-lecture@champion"
model = mlflow.pyfunc.load_model(uri)
v = client.get_model_version_by_alias(
    "electricity-mt158-lecture", "champion")
log(v.version)    # record what was used
```

Moving champion is a team decision

- Tests pass and the selection rule agrees
- A second person reviews the change
- Old and new version are recorded

Moving an alias changes the next load. A running service keeps its model until it reloads.

From a notebook model to a model others can use

Hand a model over only when every item is true.

✓ Training runs headless from a committed revision — Part 1	✓ Model logged with signature and dependencies — MLflow
✓ Environment locked; data version identifiable — Part 1	✓ Registered version with lineage; alias moved after review — MLflow
✓ Tests pass in CI: data contract, parity, reload — Part 1	✓ Input contract and feature code documented — Team
✓ Run tracked: settings, metrics, data identity, files — MLflow	✓ Known limits stated: nothing monitored yet — Part 3

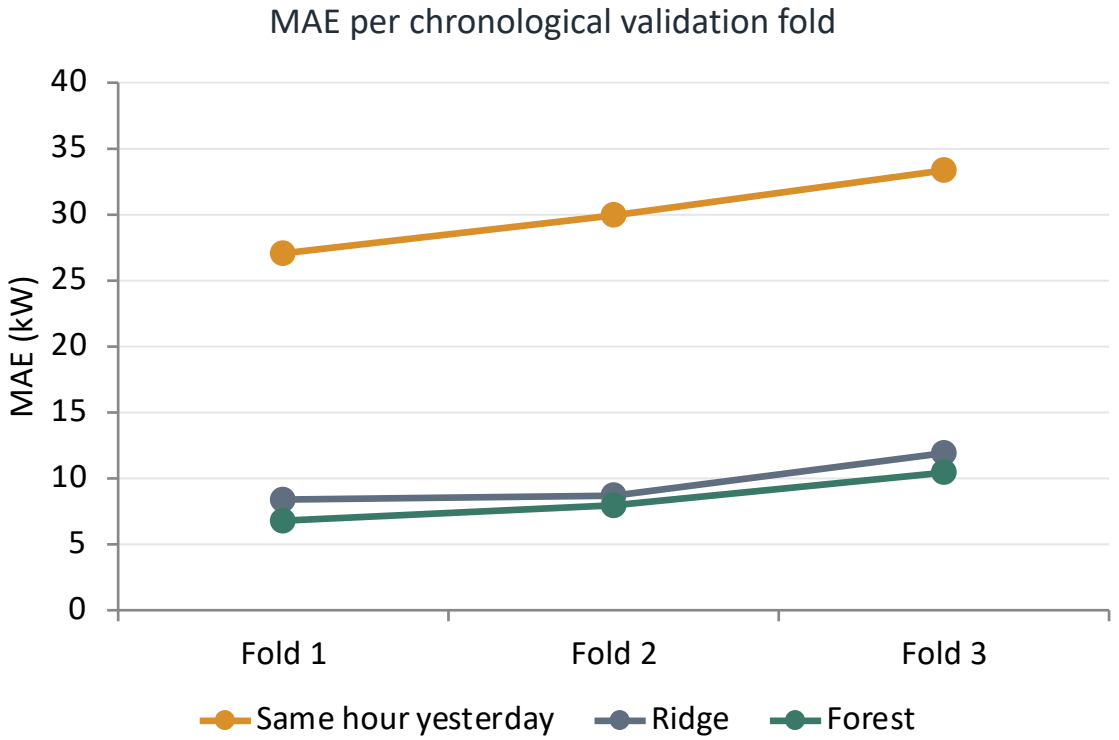
We can identify and retrieve a selected model version. How does an application use it, and how do we know it still works?

Backup: forecasting evaluation details

Development selection CV on identical target hours.

Model	MAE (kW)	RMSE (kW)
Same hour yesterday	30.124	44.606
Ridge (alpha 10)	9.663	13.501
Random forest (depth 15)	8.390	13.146

Every method has its largest error in the last window. That motivates investigation; it does not establish drift. Fold spread is descriptive, not a confidence interval.



Backup: recorded inference contract

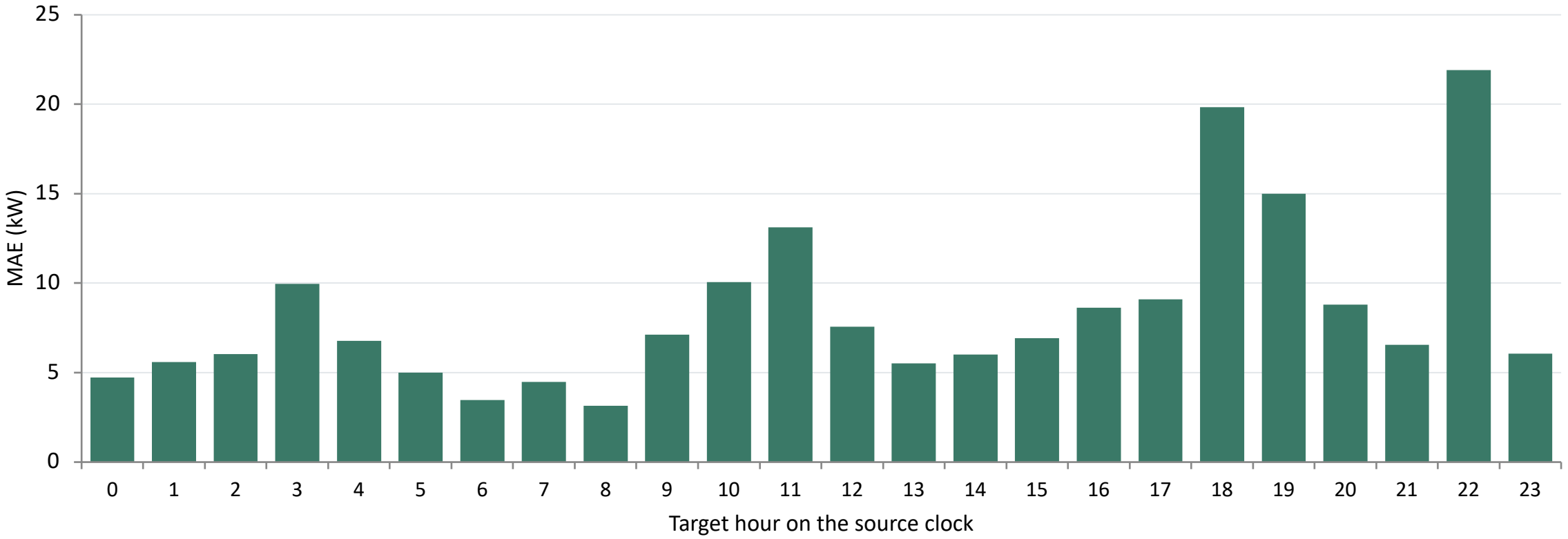
Input: at least 168 consecutive observed hours (timestamp, load_kw).

Field	Recorded value
Prediction origin	2014-11-01 00:00 (last observed hour)
Target	2014-11-01 01:00
Prediction	131.984 kW
Resolved model	models:/electricity-mt158-lecture/2 (forest)
Clock	Published Portuguese wall clock, not UTC

Shared features.build_next_features turns observed history into 29 inputs; no future value is supplied. Input validity can be checked immediately; accuracy only once the actual load arrives.

Backup: validation error by target hour

Selected forest, out-of-fold predictions from the three validation windows.



Diagnostics suggest where to investigate. They do not establish drift, a cause or an alert policy.

Backup: what the demo shows, and what it does not

Demonstrated	Tracked and nested runs, fair comparison, provenance tags, packaged and reload-verified models, registry versions, alias change and retrieval
Explained conceptually	Serving, application reload, delayed labels and monitoring (Presenter 3)
Not claimed	Shared tracking server, access control, durable artifact store, a tested fresh serving environment